NYU Tandon School of Engineering
Spring 2026, ECE 9953 (Dr. Karri) - Independent Project
Using LLMs for UVM Testbench Generation
Project Report

Student Name: Archie Gupta
Net ID: ag10276

1. Introduction

Functional verification is the dominant cost in modern ASIC and SoC development, accounting for an estimated 60–70% of total project effort. Within that budget, the Universal Verification Methodology (UVM) , a SystemVerilog class library standardized by IEEE 1800.2, is the dominant industrial framework. A complete UVM testbench (TB) for even a single design comprises roughly eleven coordinated components: sequence item, sequence, sequencer, driver, monitor, agent, scoreboard, functional coverage subscriber, environment, test, and a top-level harness. Most of this is structural boilerplate: the same factory-registration macros, the same phase methods, the same TLM hookups, instantiated across every design under test (DUT).

The pattern-driven nature of UVM testbench construction makes it a natural target for large language models. Recent work has shown LLMs can produce non-trivial RTL and assertion-style properties. The narrower question of whether LLMs can produce production-quality UVM testbenches remains open: producing eleven files with the right names and the right macros is not the same as producing a TB that elaborates, simulates, and meaningfully exercises the DUT.

This report contributes a controlled benchmark answering that question. I compare three LLMs models across 14 RTL designs of varying complexity, with a hand-written golden testbench for each design serving as the reference. All testbenches are evaluated under an identical Synopsys VCS pipeline with the same SystemVerilog Assertion (SVA) module bound to each DUT. I checked the generated output against four evaluation criteria: (i) structural conformance, (ii) compile and runtime success, (iii) functional coverage closure, and (iv) assertion coverage. I further classify all observed failure modes into a three-bucket taxonomy that scopes the cost of fixing each.

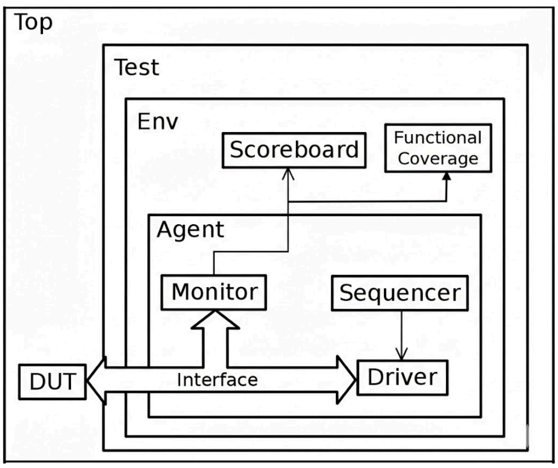

Figure 1: UVM Testbench Structure

2. Methodology

2.1 Benchmark designs

I chose fourteen RTL designs spanning three difficulty tiers. Combinational designs: 4:1 multiplexer, 3-to-8 one-hot decoder, 2-bit comparator, 4-bit ripple-carry adder, 8-bit ALU (8 opcodes), and 8-bit right barrel shifter. Sequential designs: 4-bit up/down counter, 8-bit universal shift register (USR), four-state traffic-light FSM, round-robin arbiter, and 8-entry synchronous FIFO. Bus-protocol slaves: APB3 memory slave, AHB-Lite memory slave and AXI4 memory slave. For each design I wrote a complete UVM testbench (the reference), bound a SystemVerilog Assertion module to the DUT, and authored both directed and randomised stimulus.

2.2 Models and generation pipeline

Three LLMs were evaluated: o4-mini, Gemini Pro 3.1 Preview, and Claude Opus 4.6. A Python driver loads the RTL/Specification, builds a structured prompt that lists the required UVM sections and embeds the DUT module, calls the model through a unified API gateway, and parses the response into the eleven UVM files plus a top-level harness, a top-level Makefile, and a filelist. Each (model, design) pair was attempted once; failing pairs were retried up to three additional times with the same prompt to bound retry-recovery rates.

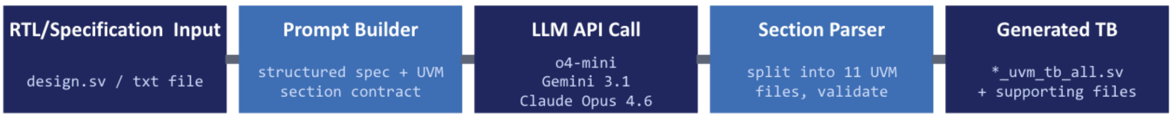

Figure 2: Framework for LLM-Assisted TB Generation

2.3 Evaluation

All testbenches , both LLM-generated and golden , were compiled and simulated under Synopsys VCS with identical compile flags and the same bound SVA module. I measured: (i) structural conformance , whether all eleven required UVM sections are produced; (ii) compile and runtime success, recording every manual edit required to reach a clean elaboration and a non-hanging simulation; (iii) functional coverage, reading the coverage report to compute variable-bin closure, cross-bin closure, and total covered bins; and (iv) assertion coverage, recording per-property attempts, matches, and pass/fail status. Compile failures were tagged with one of 30 issue classes grouped into three buckets (prompt-fixable, architectural, model-inherent). On the coverage side, I additionally classified each design's outcome by which testbench is best on three sub-axes: best by raw score, best from a testing point of view (depth of model and discipline), and best on assertion coverage (stimulus density and pass rate).

3. Results

3.1 Using LLMs to close coverage gaps using directed testing

First-pass simulation of any of the 14 testbenches in this study, including the hand-written golden testbenches, rarely produces 100% coverage closure. Random stimulus tends to hit the easy interior of the coverage space and miss boundary bins, asymmetric crosses, and temporally-conditioned cover properties (back-to-back transitions, load-then-hold sequences, write-when-full guards). The conventional response is for the verification engineer to read the URG report by hand, identify the uncovered bins, and write a directed sequence that targets each gap. I instead used the LLM as the closure engine.

3.1.1 Deepdive into Adder - Using LLMs to close coverage gaps

Metric	Result	Notes
User SVA assertions	**32 / 32 passing**	All full adder and top-level checks passing --- 0 failures
Coverpoint (Variables)	**34 / 34 bins = 100.00%**	All cp_a, cp_b, cp_cin bins fully covered
Cross coverage	**410 / 512 bins = 80.08%**	102 cross bins uncovered (19.92% gap)
Total stimuli driven	832 items	Good volume for coverage closure

*Figure 3: **Adder** - Initial Coverage*

Metric	Run 1 (Random)	Run 2 (Directed)	Combined
Stimuli count	832	104	**936**
User assertions	32/32 ✓	32/32 ✓	**32/32 ✓**
Coverpoints	34/34 (100%)	34/34 (100%)	**34/34 (100%)**
Cross coverage	410/512 (80.08%)	104/512 (20.31%)	**512/512 (100%)**
Uncovered bins	102	408	**0**

*Figure 4: **Adder** - Using LLMs to generate directed testing to close coverage gaps*

Figure 3 and figure 4 show the progression on how LLMs are used to close the coverage gaps. In this design, which is the adder, random stimulus and 800 test cases were still not enough to achieve a 100% coverage. Using LLMs to analyze the VCS report, and then generating directed tests, led to the closure of this gap.

3.1.2 Deepdive into Universal Shift Register - Using LLMs to close coverage gaps

Metric	Result	Notes
User SVA assertions	**5 / 6 passing**	ASSERT_LOAD_THEN_HOLD has 2 incomplete --- needs review
Cover properties	**5 / 6 matching**	COVER_LOAD_THEN_HOLD uncovered --- sequence not exercised
Coverpoint (Variables)	**4 / 4 bins = 100.00%**	All control modes (00, 01, 10, 11) covered
Total stimuli driven	176 items	180 clock cycles including reset

*Figure 5: **USR** - Initial Coverage*

Metric	Run 1 (Random)	Run 2 (Directed)	Combined
Stimuli count	176	64	**240**
Clock cycles	180	65	**245**
User assertions	5/6 passing	6/6 passing	**6/6 passing ✓**
Cover properties	5/6 matching	6/6 matching	**6/6 matching ✓**
Control coverage	4/4 (100%)	4/4 (100%)	**4/4 (100%)**
Assertion failures	0	0	**0**

*Figure 6: **USR** - Initial Using LLMs to generate directed testing to close coverage gaps*

Figure 5 and figure 6 show the progression on how LLMs are used to close the assertion gaps. In this design, which is the Universal Shift Register, random stimulus and 180 test cases were not enough to achieve a 100% assertion coverage. Using LLMs to analyze the VCS assertion report, and then generating directed tests to trigger the assertion, led to the closure of this gap.

3.1.3 Deepdive into Round Robin Arbiter - Using LLMs to close coverage gaps

Metric	Result	Notes
User SVA assertions	**12 / 14 passing**	2 assertions failing (10 failures total)
Cover properties	**8 / 8 matching**	All arbiter behaviors observed
Coverpoint Variables	**21 / 21 bins = 100.00%**	All req patterns + grant patterns covered
Cross Coverage	**32 / 60 bins = 53.33%**	28 req×grant combinations uncovered
Total clock cycles	499 cycles	483 operational + 16 reset cycles

Figure 7: ***Arbiter*** *- Initial Coverage*

Metric	Result	Notes
User SVA assertions	**14 / 14 passing**	All assertions covered — 0 failures
Coverpoint (Variables)	**21 / 21 bins = 100.00%**	All 3 coverpoints fully covered
Cross coverage	**32 / 60 bins = 53.33%**	28 uncovered cross bins (cx_req_grant)
Total stimuli driven	499 items	483 active cycles (rst deasserted)

Figure 8: ***Arbiter*** *- Assertion Closure using LLMs*

Metric	Result	Notes
User SVA assertions	11 / 11 passing	All user-defined assert properties are covered; 0 failures, 0 uncovered.
Assertion cover properties	8 / 8 matched	Every user cover property recorded at least one match.
Functional coverpoints	21 / 21 bins = 100.00%	cp_req, cp_grant, and cp_rst are fully covered.
Cross coverage	32 / 32 bins = 100.00%	Illegal req/grant combinations are excluded; every legal cross bin was hit.
UVM/internal uncovered assertions	4 library assertions	These are UVM/package internals, not DUT checks, and can be excluded from sign-off review.

Figure 9: ***Arbiter*** *- Coverage Closure using LLMs*

Figures 7, 8 and 9 show the incremental progression of using LLMs to close both the assertion gaps as well as the coverage gaps, for the round robin arbiter DUT.

Three patterns emerge across the closure runs. First, the LLM is most effective on designs whose uncovered bins are reachable through a small, well-described modification to the constraint solver, widening a value range, adding a randc cycling order, or specifying a directed (a, b, cin) tuple list. Second, the LLM is also effective at proposing modelling changes the human author had missed.

Design	Pass 1 score	Final score	LLM passes added	Closure mechanism
Mux	65.28%	**100.00%**	1	Constraint widening on input data values
Decoder	100.00%	100.00%	0	Closed organically (small input space)
Comparator	100.00%	100.00%	0	Closed organically
Adder	80.08%	**100.00%**	1	Directed sequence targeting 102 uncovered (a, b, cin) bins
ALU	100.00%	100.00%	0	Closed organically (opcode-only model)
Barrel shifter	100.00%	100.00%	0	Closed organically (shift-amount-only model)
Counter	—	100%	1	Coverage subscriber missing in original TB; LLM added covergroup spec
USR	83.33% (assertion coverage)	**100.00%**	1	Directed load-then-hold sequence to fire COVER_LOAD_THEN_HOLD
FSM	100.00%	100.00%	0	Closed organically
Sync FIFO	100.00%	100.00%	0	Closed organically; LLM suggested data/transition extensions
APB	100.00%	100.00%	0	Closed organically across 3 covergroups
Arbiter	<100%	**100.00%**	1	LLM added illegal_bins for impossible req/grant pairs (60 → 32 legal cross)
AHB-Lite	70.56%	70.56%	1	LLM added INCR4 burst sequence; IDLE/BUSY HTRANS still uncovered
AXI4	65.52%	65.52%	1	LLM authored back-pressure pass; 8 stability bins remain unreachable (DUT architecture)

Table 1: LLM-driven coverage closure outcomes per design. First-pass coverage score, final score after LLM-driven feedback iterations, number of additional LLM passes added, and the closure mechanism the model proposed.

3.2 Structural conformance

Table 2 reports first-attempt structural conformance for all three models. Claude Opus 4.6 and o4-mini tied at 93% (13 of 14 designs each); Gemini Pro 3.1 Preview reached only 29%. Failure modes correlate with output-token budget rather than model size:

	adder	ahb	alu	apb	arbiter	axi	barrel	comp	counter	decoder	fifo	fsm	mux	syncfifo
o4-mini	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Gemini 3.1	✓	✗	✗	✗	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
Claude Opus 4.6	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓

Figure 10: Breakdown of Design TB vs LLM Model for first pass success

Gemini 3.1 fails by truncation on the long protocol-design TBs, which was resolved by increasing the tokens permitted for the attempt from 16k to 63k. Retries recovered structural conformance for every failure on at least one of three attempts, indicating the underlying capability is present but determinism is the gap.

Model	Pass rate	Designs passed	Primary failure mode
o4-mini	93%	13 / 14	AHB only (incidental compile error)
Gemini Pro 3.1	29%	4 / 14	Heavy output truncation
Claude Opus 4.6	93%	13 / 14	AXI only (output budget)

Table 2: First-attempt structural conformance. Percentage of 14 designs for which the model produced all eleven required UVM sections in a single attempt. Passes/failures by design and the dominant failure mode are listed.

3.3 Compile and runtime success

Zero of three models produced a testbench that compiled and ran cleanly under VCS without manual intervention. Every output required at least one human edit; some required dozens.

3.3.1 Deepdive into Counter - Compilation Issues

Issue	Error Category	Fix
missing `uvm_object_utils(counter_seq_item) and logic [3:0] dout;	A	Add factory macro + output field
body declared as function	A	Change to virtual task body()
start(item) used incorrectly	C	Replace with start_item(item) / finish_item(item)
invalid uvm_test_top.clock cross-module reference	A	Removed
coverage class samples vif.cb_mon.* (compile fail)	A	Replace with transaction-based coverage
argument name mismatch in coverage write()	C	Rename item to t
sequence declaration / assignment combined → syntax error	C	Split into two statements
run declared as function	A	Change to virtual task run_phase(uvm_phase phase)
starting sequence inside function	B	Move into run_phase task
driver's seq_item_port not connected to sequencer	B	Add driver.seq_item_port.connect(sequencer.seq_item_export)

*Figure 11: **Counter** TB generated using Claude - Compile Errors and their Categories*

I classified the resulting issue catalogue (Table 3) into three buckets: A , Prompt-Fixable (15 issue classes, hundreds of instances), B , Architectural (6 issue classes), and C , Model-Inherent (9 issue classes). Bucket A dominates by frequency: missing `uvm_macros.svh`, missing factory-registration macros, concrete (non-virtual) interface declarations, run_phase declared as a function rather than a task, and coverpoints that sample the virtual interface instead of the transaction. Bucket B captures architectural misunderstandings: components created in the wrong UVM phase, missing connect_phase wiring, and forgotten objections that cause PH_TIMEOUT or simulation hangs. Bucket C is the residual that prompting cannot close: hallucinated UVM APIs (sequencer.start(), driver.start()), missing transaction output fields, and wrong scoreboard reference models that compile cleanly but flag every transaction with [SCB_FAIL]. Gemini 3.1 was the model most prone to wrong-reference-model bugs, which are the hardest to catch because the testbench looks right and runs cleanly while checking the wrong thing.

Bucket	Count	Primary cause	Prompt-fix yield
A - Prompt-Fixable	15 classes	Boilerplate forgetfulness , recurring scaffolding mistakes	High: prompt templates and a do/don't reference card close most of this
B - Architectural	6 classes	UVM phase / connection model not fully internalised	Medium: a worked-out skeleton testbench in the prompt
C - Model-Inherent	9 classes	Hallucination, comprehension limits, wrong reference models	Low: needs better models, retrieval grounding, or post-hoc validation

Table 3: Compile-failure taxonomy. The 30 observed issue classes group into three buckets with different fix-cost profiles. Bucket A dominates by frequency and is closeable with prompt engineering; bucket C is residual model-capability gap.

3.4 Functional and assertion coverage

For the coverage analysis I held the model set fixed at three: o4-mini, Gemini Pro 3.1, and Claude Opus 4.6, compared per-design against the golden original. Across all 14 designs I logged six metrics per testbench: overall coverage score, user SVA assertion pass rate, cover-property hit count, coverpoint variable closure, cross-coverage closure, and total clock cycles per assertion (a stimulus-density proxy). I then classified each design by which testbench wins on three sub-axes: best by score, best from a testing point of view, and best on assertion coverage. Claude Opus 4.6 wins the testing-point-of-view sub-axis on 11 of 14 designs by virtue of richer covergroup models , output observability, ignore_bins discipline, transition coverpoints, and user-defined cross-bins formalising protocol invariants , combined with the highest stimulus density on most designs. The original testbench wins it on the four protocol designs (APB, AHB, AXI, arbiter), where it is the only TB to use protocol-specific ignore_bins, HBURST coverage, FSM-traversal stimulus tuning, and back-to-back transition crosses. o4-mini is competitive on simple designs but fails on protocol designs (APB at 40.62%, arbiter at 21.74%), with stimulus density 10 to 100 times below Claude on every design. Gemini 3.1 produced the only complete coverage-wiring failure.

3.4.1 Deepdive into Mux - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	100.00 %	65.28 %	100.00 %
User SVA assertions	9 / 11 passing	8 / 10 passing	8 / 11 passing	16 / 18 passing
Cover properties	7 SVA props ✓	7 SVA props ✓	7 SVA props ✓	7 SVA props ✓
Coverpoint variables	68 / 68 = 100 %	6 / 6 = 100 %	12 / 12 = 100 %	34 / 34 = 100 %
Cross coverage	196 / 196 = 100 %	8 / 8 = 100 %	7 / 32 = 21.88 %	80 / 80 = 100 %
Total clock cycles / assert	2,116	88	414	877

*Figure 12: **Mux** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 12 shows the comparison of the results of the functional coverage and assertion for the mux design. Here we can see o4-mini and Gemini 3.1 Pro lacks with respect to the number of coverpoint point variables being added to the coverage class. Even though both o4-mini and Claude show 100% coverage, the number of coverpoint variables and cross coverage is lower than the original testbench, which might indicate insufficient testing of the RTL code.

3.4.2 Deepdive into Decoder - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	7.03 %	70.83 %	78.41 %
User SVA assertions	16 / 18 passing	8 / 17 passing	15 / 17 passing	15 / 17 passing
Cover properties	all 14 SVA props attempted	7 / 14 unfired	all attempted	all attempted
Coverpoint variables	8 / 8 = 100 %	2 / 72 = 2.78 %	16 / 16 = 100 %	35 / 35 = 100 %
Cross coverage	— (no cross)	— (no cross)	8 / 64 = 12.50 %	12 / 88 = 13.64 %
Total clock cycles / assert	1,292	34	918	3,638

*Figure 13: **Decoder** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 13 shows the comparison of the results of the functional coverage and assertion for the decoder design. Here we can see o4-mini does not reach an acceptable overall score - due to low coverage in both assertions as well as coverpoint variables. Even though both Gemini 3.1 Pro and Claude generated cross coverage, the nature of the RTL design is such that these bins would never be hit. This leaves room for improvement for the concept of ignore_bins.

3.4.3 Deepdive into Adder - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100 % (random-only: 80.08)	~70.83 %	95.24 %	100.00 %
User SVA assertions	34 / 36 passing	34 / 36 passing	34 / 36 passing	34 / 36 passing
Cover properties	all attempted	all attempted	all attempted	all attempted
Coverpoint variables	34 / 34 = 100 %	10 / 38 (split CGs)	13 / 13 = 100 %	18 / 18 = 100 %
Cross coverage	512/ 512 = 80.08 %	0 / 0 (no cross)	10 / 13 = 76.92 %	107 / 107 = 100 %
Total clock cycles / assert	7,439	89	933	9,092

*Figure 14: **Adder** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 14 shows the comparison of the results of the functional coverage and assertion for the adder design. Here we can see o4-mini lacks with respect to the percentage of coverpoint point variables being hit. Even though Claude shows 100% coverage, the number of coverpoint variables and cross coverage is lower than the original testbench.

3.4.4 Deepdive into Comparator - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	68.75 %	97.92 %	90.91 %
User SVA assertions	19 / 21 passing	17 / 22 passing	18 / 20 passing	18 / 20 passing
Cover properties	all 17 attempted	GT_* never attempted	all attempted	all attempted
Coverpoint variables	8 / 8 = 100 %	5 / 6 = 83.33 %	14 / 14 = 100 %	23 / 23 = 100 %
Cross coverage	16 / 16 = 100 %	2 / 8 = 25 %	14 / 16 = 87.50 %	52 / 88 = 75.00 %
Total clock cycles / assert	24,412	30	332	1,125

*Figure 15: **Comparator** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 15 shows that the original testbench has the dominant stimulus density (24K attempts); cp_a × cp_b cross which saturates with random cases. o4-mini never produced a > b case, which resulted in 3 case asserts (GT_HIGH, GT_EQ, GT_LT) never attempted; agreaterb output bin uncovered. Gemini 3.1 has 2 missing cross bins that are random-stimulus gaps. Claude had composite cp_result with illegal_bins for non-one-hot outputs; 3-way cx_full cross has unreachable bins (should have been illegal_bins).

3.4.5 Deepdive into ALU - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	77.78 %	91.38 %	92.75 %
User SVA assertions	22 / 24 passing	22 / 24 passing	22 / 24 passing	22 / 24 passing
Cover properties	all attempted	low SUB / SHR (1–2×)	all attempted	all attempted
Coverpoint variables	8 / 8 = 100 %	12 / 14 = 85.71 %	26 / 26 = 100 %	35 / 35 = 100 %
Cross coverage	— (no cross)	— (no cross)	62 / 200 = 31.00 %	160 / 192 = 83.07 %
Total clock cycles / assert	~480	~210	~2,700	~5,640

*Figure 16: **ALU** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 16 shows us the coverage analysis for the ALU. Here we see the original is hollow; there is no operand or flag coverage. o4-mini has bins that are never hit; 1-bit-flip bugs could go undetected. Gemini 3.1 generated signed-arithmetic-aware operand bins and the cross drops to 31 % from over-expansion. Claude is the only TB with output coverage and 4 flag×opcode crosses; finer 8-bin operand model which targets every signed corner.

3.4.6 Deepdive into Barrel Shifter - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	66.67 %	61.67 %	~95.7 %
User SVA assertions	7 / 9 passing	7 / 9 passing	7 / 9 passing	7 / 9 passing
Cover properties	all attempted	ZERO / MAX fired 1×	ZERO / MAX 7×	ZERO / MAX 91–92×
Coverpoint variables	8 / 8 = 100 %	4 / 5 = 80 %	11 / 13 = 84.6 %	35 / 35 = 100 %
Cross coverage	— (no cross)	2 / 6 = 33.3 %	10 / 40 = 25 %	136 / 152 = 89.5 %
Total clock cycles / assert	198	50	260	2,523

*Figure 17: **Barrel Shifter** - Original TB vs LLM TBs Functional Coverage and Analysis*

Here in figure 17, we see that the original testbench has no cross coverage defined. In the o4-mini testbench, only 2 of 8 shift values declared as bins; x=0x00 corner never hit - therefore having the lowest stimulus. Gemini 3.1 declared corner-pattern bins (0xA5, 0x5A) which were never organically hit by random, resulting in higher number of coverage points, but a lower coverage percentage.

Claude: dedicated covergroup crosses covers most of the single-bit input × every shift amount. Zero/Max indicates the number of times the boundaries of the DUT.

3.4.7 Deepdive into Counter - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	N/A (no covergroup)	100.00 %	99.31 %	100.00 %
User SVA assertions	8 / 12 passing	7 / 9 passing	7 / 9 passing	7 / 9 passing
Cover properties	all 6 hit	all 6 hit (1-2× each)	all 6 hit	all 6 hit (high density)
Coverpoint variables	0 (no covergroup)	20 / 20 = 100 %	22 / 22 = 100 %	14 / 14 = 100 %
Cross coverage	—	— (no cross)	35 / 36 = 97.22 %	24 / 24 = 100 %
Total clock cycles / assert	3,840	258	576	1,368

*Figure 18: **Counter** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 18 shows the original testbench coverage is weakened by a commented-out subscriber and two never-executed sequence-body assertions. o4-mini brute-forces all 16 dout values but adds little intent, with WRAP_UP/DOWN firing only once. Gemini has the best temporal model via 15→0 and 0→15 transition bins, while Claude has the strongest boundary/cross coverage and highest cover-property density.

3.4.8 Deepdive into Universal Shift Register - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	78.91 %	57.14 %	100.00 %
User SVA assertions	7 / 9 passing	7 / 9 passing	7 / 9 passing (1 incomplete)	7 / 9 passing
Cover properties	all 6 hit	all 6 hit (low density)	all 6 hit	all 6 hit (5–80× more)
Coverpoint variables	4 / 4 = 100 %	18 / 72 = 25 %	10 / 13 = 75 %	37 / 37 = 100 %
Cross coverage	— (no cross)	— (no cross)	1 / 28 = 3.57 %	19 / 19 = 100 %
Total clock cycles / assert	390	78	696	3,384

*Figure 19: **USR** - Original TB vs LLM TBs Functional Coverage and Analysis*

Figure 19 shows a different outcome with respect to the functional coverage and assertion for the Universal Shift Register design. Here we can see o4-mini continues to lack in percentage of coverpoint point variables being hit, and there is no cross coverage defined. Claude shows 100% coverage, on a higher number of coverpoint variables and cross coverage than the original testbench.

3.4.9 Deepdive into FSM- Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	85.19 %	0.00 %	60.00 % main (100 % outputs)
User SVA assertions	8 / 10 passing	9 / 11 passing	8 / 10 passing	8 / 10 passing
Cover properties	all 6 hit	all 6 hit	all 6 hit	all 6 hit (highest density)
Coverpoint variables	8 / 8 = 100 %	6 / 6 = 100 %	0 / 10 = 0 %	14 / 16 (multi-CG)
Cross coverage	— (no cross)	5 / 9 = 55.56 %	0 / 7 = 0 %	11 / 48 = 22.92 %
Total clock cycles / assert	315	154	539	707

*Figure 20: **FSM** - Original TB vs LLM TBs Functional Coverage and Analysis*

In figure 21, we look at the FSM. The original hits 100% on a shallow 3-coverpoint/no-cross model, with reset barely exercised. o4-mini's score is dragged down by unreachable green/yellow cross bins with no ignore_bins. Gemini's coverage fails from TB wiring defects, though assertions still fire. Claude is strongest, covering the full FSM cycle and illegal state encodings.

3.4.10 Deepdive into SyncFIFO - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	72.40 %	100.00 %	97.50 %
User SVA assertions	11 / 13 passing	10 / 13 passing	11 / 13 passing	11 / 13 passing
Cover properties	6 / 6 hit	5 / 6 hit (NO_WRITE_WHEN_FULL=0)	6 / 6 hit	6 / 6 hit
Coverpoint variables	10 / 10 = 100 %	15 / 68 = 22 %	8 / 8 = 100 %	23 / 23 = 100 %
Cross coverage	— (no cross)	— (no cross)	18 / 18 = 100 %	27 / 33 = 81.82 %
Total clock cycles / assert	1,350	200	1,760	3,060

*Figure 21: **SyncFIFO** - Original TB vs LLM TBs Functional Coverage and Analysis*

In figure 21, we analyze the coverage of the synchronous FIFO design. Here I note that the original testbench has the best stimulus on safety-critical guard-band corners - NO_WRITE_WHEN_FULL. The o4-mini testbench never drove FIFO into the write-when-full corner — that cover property has 0 matches, and the percentage of the coverpoint variables hit were also less than 25%. The Gemini 3.1 testbench declared ignore_bins on the very corner that should be covered, which is a coverage-model misjudgment. The Claude generated TB was the only TB to cover internal count + both wr_data and rd_data; user-defined cross-bins explicitly target what gem31 ignored.

3.4.11 Deepdive into APB - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 %	40.62 %	100.00 %	97.78 %
User SVA assertions	11 / 14 passing	9 / 14 passing	10 / 14 passing	11/14 passing
Cover properties	7 / 7 hit	5 / 7 hit	6 / 7 hit	7 / 7 hit
Coverpoint variables	19 / 19 = 100 %	7 / 68 = 10.29 %	9 / 9 = 100 %	15 / 16 = 96 %
Cross coverage	4 / 4 = 100 %	8 / 128 = 6.25 %	10 / 10 = 100 %	20 / 20 = 100 %
Total clock cycles / assert	1,110	20	126	1,162

*Figure 22: **APB Protocol** - Original TB vs LLM TBs Functional Coverage and Analysis*

From figure 22, we see that the original testbench has the strongest FSM traversal and uniquely covers back-to-back R/W transitions. o4-mini is too shallow, with weak address coverage and several unfired checks. Gemini has a clean address-validity model but misses ACCESS→IDLE. Claude has the richest coverage model, with 15 coverpoints, 20 named crosses, and protocol illegal_bins.

3.4.12 Deepdive into Round Robin Arbiter - Functional Coverage and Assertion Coverage Comparison

Metric	Original (3 runs)	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	100.00 % (run-3)	21.74 %	88.39 %	92.45 %
User SVA assertions	passing (caught real fails)	low pass rate	passing	85.71 % (best LLM)
Cover properties	all hit (1428×)	4 cover-props unfired	all hit	all hit (281×)
Coverpoint variables	21 vars + 32 crosses	low closure	good w/ illegal_bins	13 cps incl. per-bit + cg_sva
Cross coverage	32 / 32 = 100 % w/ ignore_bins	very low	no ignore_bins on impossible pairs	no ignore_bins on impossible pairs
Total clock cycles / assert	1,497	9	107	290

*Figure 23: **Arbiter** - Original TB vs LLM TBs Functional Coverage and Analysis*

Using figure 23, we can draw the conclusion that the original is strongest here, with protocol-correct ignore_bins for all impossible req/grant cases and 3× more stimulus than any LLM TB. o4-mini fails badly with 21.74% coverage and many unfired checks. Gemini shows protocol awareness but loses coverage on cross gaps. Claude is structurally richest, but this is the first design where Original clearly sweeps every category.

3.4.13 Deepdive into AHB - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	~87 %	80.00 %	91.67 %	78.72 %
User SVA assertions	11 / 14 passing	15 / 19 passing	11 / 14 passing	33 / 36 passing (best %)
Cover properties	9 / 9 hit	8 / 9 hit	9 / 9 hit	9 / 9 hit (16× err-paths)
Coverpoint variables	~38 bins (3 CGs)	8 / 10 = 80 %	10 / 10 ≈ 100 %	61 bins (word_idx, alignment)
Cross coverage	~33 cross bins	6 cross bins	12 cross bins	30 cross bins
Total clock cycles / assert	240 max / 720 sum (3 runs)	57	64	231

*Figure 24: **AHB Protocol** - Original TB vs LLM TBs Functional Coverage and Analysis*

Using figure 24, we see that for the AHB protocol, the original is only TB to cover HBURST (6 burst types) + HTRANS state-transition cross; extensive ignore_bins discipline. o4-mini never exercised IDLE/BUSY HTRANS or deselected slave, and had the lowest stimulus. Gemini 3.1 was compact size/write/trans/resp model; missing illegal_bins for err_sz × okay (impossible), and Claude had 21 sequence-body asserts (highest success rate); 32-bin word_idx + 12-bin alignment (richest model).

3.4.14 Deepdive into AXI - Functional Coverage and Assertion Coverage Comparison

Metric	Original	o4-mini	Gemini 3.1 Pro	Claude Opus 4.6
Overall coverage score	~97.78 %	88.54 %	95.24 %	~86.16 %
User SVA assertions	19 / 29 passing	27 / 37 passing	23 / 33 passing	22 / 30 passing
Cover properties	9 / 9 hit	9 / 9 hit	9 / 9 hit (BVALID 6476×)	9 / 9 hit
Coverpoint variables	39 bins (5 CGs)	12 bins ≈ 88.54 %	13 bins ≈ 95.24 %	64 bins ≈ 88 %
Cross coverage	~25 cross bins	14 cross bins	12 cross bins	36 cross bins
Total clock cycles / assert	1,023	267	6,509	1,036

*Figure 25: **AXI Protocol** - Original TB vs LLM TBs Functional Coverage and Analysis*

Finally from figure 25, we draw the conclusion that the original has the strongest protocol-focused model, with per-channel coverage, a dedicated error covergroup, and byte-error ignore_bins. o4-mini has decent ignore_bins discipline but misses size_2B. Gemini drives the most stimulus by far. Claude is broadest structurally, but loses coverage on cross bins to reserved-burst impossibilities that should be illegal_bins.

Table 4 summarises the per-design verdict on each sub-axis. The pattern is clear: Claude dominates the testing-point-of-view sub-axis on every non-protocol design, the original dominates the protocol designs, and assertion coverage is split between Claude (on the simpler designs, by sheer stimulus density) and the original (on designs where careful corner-case stimulus matters more than volume, sync FIFO's no-write-when-full corner, APB's ACCESS-to-IDLE FSM transition, and the arbiter).

Design	Best by score	Best from a testing PoV	Best assertion coverage
Mux	tied: orig / o4-mini / claude	Claude	Claude
Decoder	Original	Claude	Claude (close to Original)
Adder	Claude (organic)	Claude	Claude
Comparator	Original	Claude	Original (24K attempts)
ALU	Original (hollow)	Claude	Claude
Barrel shifter	Original (hollow)	Claude	Claude
Counter	tied: o4-mini / claude	Gemini 3.1 (transition bins)	Claude
USR	tied: orig / claude	Claude	Claude
FSM	Original	Claude (full-cycle seq.)	Claude
Sync FIFO	tied: orig / gem31	Claude	Original (corner stimulus)
APB	tied: orig / gem31	Claude	Original (FSM transitions)
Arbiter	Original (3 runs)	Original	Original (caught 10 fails)
AHB	Gemini 3.1	Original	Claude
AXI	Original	Original	Original

Table 4: Per-design coverage winners. For each design, the testbench that wins on each of three coverage sub-axes , raw score, depth of testing model, and assertion stimulus density and pass rate. Original = hand-written golden TB.

4. Future Work

First, skeleton-based prompt engineering providing a worked-out UVM testbench skeleton in the prompt with all eleven sections stubbed and TLM hookups pre-wired, can be used to close most of the prompt-fixable Category-A bucket without any change to the model, and is directly measurable against Table 2. Second, coverage-driven feedback loops which parse URG output programmatically, hand the structured summary to the LLM, accept proposed sequences only if they compile, close at least one previously-uncovered bin, and do not regress any existing bin, can be used to further automate the process of UVM testbench generation. Third, integration with LLM-generated assertions lets the LLM propose the bound SVAs as well as the testbench, a higher-stakes question that the present study deliberately controls for by using a fixed assertion set, and one that requires a separate reference-oracle infrastructure to evaluate honestly. Fourth, localised training using industry UVM testbenches, to pre-condition the LLM targets the failures that prompting alone cannot close.

5. Conclusion

I benchmarked three LLMs against hand-written golden testbenches across 14 RTL designs and four orthogonal evaluation axes. Three findings stand out. First, structural conformance is largely solved: Claude Opus 4.6 and o4-mini both produce eleven correctly-named UVM sections on 13 of 14 designs in a single attempt, and retry recovery closes most remaining structural failures. Second, structural correctness is necessary but not sufficient: zero of three models produced a testbench that elaborated and simulated cleanly without manual intervention, and the bulk of compile failures sit in a prompt-fixable bucket of fifteen recurring scaffolding mistakes. Third, the verification-quality gap is sharper than the structural one. Claude produces the structurally richest covergroups on most designs, but on the four bus-protocol designs , and on the only design that exposed a real bug, the hand-written original retains a meaningful lead through protocol-aware ignore_bins discipline and corner-case stimulus tuning. LLMs are credible UVM first-draft authors today; they are not yet competent verification engineers. The natural next steps are skeleton-grounded prompts to close bucket A, coverage-driven feedback loops to harden bucket B, and retrieval-grounded scoreboard reference models to chip away at bucket C.